Distributed Systems for Scaling Large Language Models

ARYAN BALDUA and SATHVIK CHILAKALA, California Polytechnic State University, USA

Large Language Models (LLMs) have moved rapidly from research prototypes to core infrastructure for modern AI applications. At the same time, their sheer scale has made both training and deployment a serious distributed systems challenge. Today's transformer-based models often contain billions, and increasingly hundreds of billions, of parameters. Training models at that scale demands enormous computational throughput, substantial memory capacity, and careful coordination across clusters of GPUs, because no single machine can supply enough compute, memory, or bandwidth on its own.

For that reason, scaling LLMs is no longer just a machine learning problem. It is just as much a systems problem, shaped by how computation is divided, how memory is handled, how communication is coordinated, and how failures are managed across distributed hardware. In practice, performance depends not only on the model architecture itself, but also on whether the underlying system can keep accelerators busy while limiting synchronization overhead, memory duplication, and network bottlenecks.

This survey looks at the distributed systems techniques that make large-scale LLM training and serving possible. We organize the literature around seven connected themes. First, parallelism strategies such as data, tensor, pipeline, and hybrid parallelism divide computation across many GPUs. Second, memory-focused optimizations, including activation checkpointing, ZeRO-style sharding, and optimizer offloading, reduce the memory demands of training large models. Third, communication optimizations improve the efficiency of collectives and other data movement operations, which increasingly dominate performance at scale. Fourth, fault tolerance and resilience mechanisms address a practical reality: long-running jobs on large clusters must contend with stragglers, node failures, and other system disruptions. Fifth, sparse Mixture-of-Experts systems introduce new routing, scheduling, and communication challenges. Sixth, inference serving systems focus on deploying trained models efficiently under strict latency and throughput constraints. Seventh, production infrastructure and automated parallelism bring these techniques together into end-to-end training and serving stacks.

We survey 27 papers, including foundational systems such as Megatron-LM and ZeRO, large-scale training case studies such as MegaScale, recent work on fault-tolerant training such as Oobleck and Bamboo, and modern inference-serving systems including vLLM, Orca, and DistServe. Taken together, these works point to a consistent conclusion: distributed systems concerns such as workload partitioning, synchronization, memory hierarchy management, communication scheduling, and resilience are not secondary implementation details. They are central to whether LLM training and serving are practical, efficient, and scalable.

ACM Reference Format:
Aryan Baldua and Sathvik Chilakala. 2026. Distributed Systems for Scaling Large Language Models. 1, 1 (March 2026), 10 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

Authors' Contact Information: Aryan Baldua, abaldua@calpoly.edu; Sathvik Chilakala, schila@calpoly.edu, California Polytechnic State University, San Luis Obispo, California, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2026/3-ART
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Context

Large Language Models (LLMs) have expanded quickly in both scale and capability, and they now underpin many modern natural language processing applications. Transformer-based models, including GPT-style systems, often contain billions of parameters, which makes training them increasingly expensive and difficult to scale. At this point, training these models on a single machine is no longer realistic because of limits in GPU memory, compute capacity, and overall training time. Instead, researchers and practitioners depend on clusters of GPUs and distribute training across many machines. Because of this, LLM training is no longer just a machine learning challenge. It is also a distributed systems problem, where performance depends on how well computation, memory, and communication are coordinated across nodes.

This survey examines distributed systems techniques for training and serving LLMs on GPU clusters. That scope is narrow enough to keep the discussion focused, but still broad enough to cover the main system-level approaches used in practice. Prior work has explored several forms of parallelism, including tensor or model parallelism [Shoeybi et al. 2019], pipeline parallelism [Narayanan et al. 2019], and hybrid approaches that combine data, tensor, and pipeline parallelism [Narayanan et al. 2021]. Other work has focused on memory efficiency by reducing redundancy across distributed workers through sharding and optimizer-state partitioning [Rajbhandari et al. 2020, 2021]. System frameworks such as Alpa further extend these ideas by automating the coordination of such strategies at scale [Zheng et al. 2022].

More recent studies suggest that scaling LLM training across very large GPU clusters requires more than simply choosing the right parallelism strategy. It requires coordination across the entire system. MegaScale, for example, draws out practical lessons from training on more than 10,000 GPUs [Jiang et al. 2024]. Memory also remains a major bottleneck, which has led to techniques such as optimizer-state offloading [Yu et al. 2024]. At the same time, Mixture-of-Experts (MoE) models introduce new communication and scheduling challenges because only a subset of parameters is activated for each input [Liu et al. 2023; Shi et al. 2024]. Beyond training, serving trained LLMs brings its own set of distributed systems challenges, especially around memory management, request scheduling, and resource disaggregation [Kwon et al. 2023; Yu et al. 2022; Zhong et al. 2024]. Taken together, these studies show that core distributed systems concerns such as workload partitioning, synchronization, communication efficiency, and resource management are central to whether LLM training and serving are feasible and efficient.

2 Paper Selection

The papers included in this survey were selected through a targeted review of research on distributed systems support for large-scale LLM training and serving. We began by looking at publications from top-tier systems and high-performance computing venues,

with particular attention to the International Conference for High Performance Computing, Networking, Storage, and Analysis (SC). Because SC regularly publishes work on large-scale GPU clusters, distributed training, and communication-efficient systems, it provided a strong starting point for identifying papers related to LLM scaling. From those proceedings, we selected an initial set of seed papers that addressed core training challenges such as parallelization strategies [Narayanan et al. 2021; Shoeybi et al. 2019] and memory-efficient training [Rajbhandari et al. 2020].

From there, we expanded the survey corpus by reviewing the reference lists of those seed papers and using Google Scholar to identify both earlier foundational work and more recent papers that cited them. This helped us trace how key ideas developed over time, from early systems for distributed model training to newer frameworks and large-scale case studies. Since citation links alone do not necessarily indicate relevance, we limited the survey to papers that make a substantial contribution to distributed systems techniques for LLM training or serving. We also gave priority to papers published in well-established venues such as NSDI, OSDI, SOSP, MLSys, NeurIPS, and ICLR, while treating citation count as a secondary signal of impact and visibility.

Some papers take on a more central role in the survey than others. Megatron-LM and ZeRO, for example, are emphasized because they introduce core ideas in model parallelism and memory optimization that appear repeatedly in later work. Other papers, such as PipeDream and Alpa, are included to represent complementary directions, including pipeline parallelism and automated parallelization. We also extend the survey beyond training to cover inference-serving systems such as vLLM, Orca, and DistServe, reflecting the growing importance of deployment efficiency in the distributed systems literature. Together, these 27 papers form a cohesive body of work spanning dense Transformer training, memory scaling, automated parallelization, production-scale efficiency, sparse MoE training, communication optimization, fault tolerance, and inference serving.

3 Key Issues

This survey is organized around five core system-level questions that shape whether training and serving a large language model on a GPU cluster is both feasible and efficient:

(1) Parallelism strategies: How can a single training job be distributed across many GPUs while keeping idle time and synchronization overhead as low as possible?

(2) Memory-oriented optimizations: How can parameters, gradients, activations, and optimizer state fit within limited GPU memory without slowing training too much?

(3) Communication optimizations: How can the bandwidth and latency costs of synchronizing gradients and model state across hundreds or thousands of GPUs be reduced?

(4) Fault tolerance: How can long-running training jobs continue despite GPU or node failures without creating excessive recovery overhead?

(5) Inference serving: How can trained LLMs be served efficiently to many concurrent users while still meeting latency and throughput requirements?

These five questions provide the main organizational lens of the survey. Later sections extend this discussion to sparse MoE systems and production infrastructure, where multiple techniques are combined into end-to-end training and serving stacks.

3.1 Parallelism Strategies

At a high level, training follows a repeated loop: the model executes a forward pass, computes a loss, performs a backward pass to obtain gradients, and then updates its weights. Once training is distributed across many GPUs, the central challenge becomes keeping those GPUs busy with useful computation rather than letting them sit idle. A GPU may wait at synchronization points for other devices to catch up, or it may stall while the network delivers gradients, activations, or parameter shards. Because of this, the goal of a parallelism strategy is not simply to use more GPUs. It is to divide the work in a way that preserves high utilization while keeping coordination overhead manageable.

A natural baseline is data parallelism. In this approach, every GPU stores a full copy of the model and processes a different shard of the input data, typically a different portion of the batch. After computing gradients locally, the workers must synchronize so they can apply a consistent update, usually by averaging gradients through an AllReduce operation. Data parallelism is attractive because it is conceptually simple and can provide close to linear speedup at modest scale. If eight GPUs each process one-eighth of the batch, throughput can increase substantially. That simplicity made data parallelism one of the earliest and most widely adopted strategies for distributed deep learning, and systems such as Horovod helped make it much easier to deploy in practice [Sergeev and Del Balso 2018].

However, data parallelism becomes insufficient for large language models because of two major problems.

The first is memory duplication. In standard data parallelism, every GPU stores a complete copy of the model, along with gradients and optimizer state. This means the full training state must fit within the memory of a single device. For modern LLMs, that requirement quickly becomes infeasible. The problem is especially severe because training memory is not limited to model weights alone. It also includes gradients, optimizer state, and intermediate activations saved during the forward pass for use in backpropagation. As a result, even if the parameters alone barely fit on one GPU, full training may still exceed memory capacity.

The second problem is communication overhead. Each data-parallel training step requires gradient synchronization across all workers, and the cost of that synchronization grows with both model size and cluster scale. As models become larger, the gradient tensors grow larger as well. As clusters scale out, communication can consume an increasing fraction of total step time. Romero et al. show that these performance issues are shaped not only by raw bandwidth, but also by orchestration, scheduling, and buffering decisions in the training stack, all of which affect how efficiently collectives execute in practice [Romero et al. 2022]. TCCL and TACCL further show that the best collective algorithm depends on the topology of the cluster, meaning that a single default implementation is not always optimal [Li et al. 2024; Shah et al. 2023]. AIACC-Training similarly

shows that increasing concurrency in gradient communication can reduce waiting time and improve end-to-end efficiency [Lin et al. 2022].

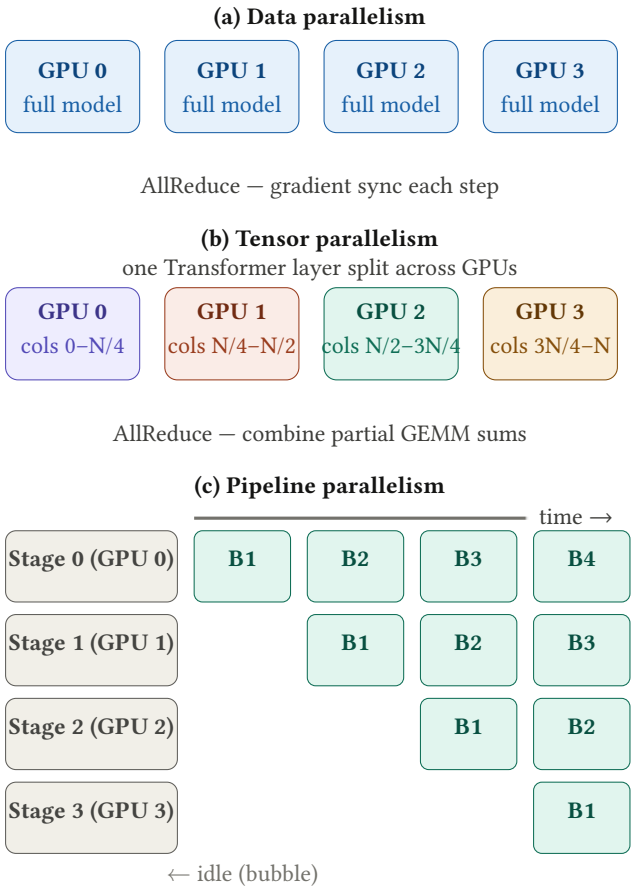

Figure 1. Three parallelism strategies for LLM training. Dashed cells = pipeline bubble (GPU idle).

Fig. 1. Three common parallelism strategies for large-scale LLM training. Data parallelism replicates the full model across GPUs and splits the data batch. Tensor parallelism partitions individual layers across GPUs. Pipeline parallelism partitions the model by layers into stages and streams microbatches through them.

Figure 1 illustrates three major parallelism strategies used in large-scale training. In data parallelism, each GPU holds a full copy of the model and processes a different data shard, with gradients synchronized across workers. In tensor parallelism, a single layer is partitioned across multiple GPUs so that each device stores and computes only part of the layer. In pipeline parallelism, the model is divided by layers into stages across GPUs, and microbatches flow through those stages in sequence.

When data parallelism is no longer enough, usually because the model cannot fit on a single GPU, training must move toward model parallelism. One important form is tensor parallelism, also called intra-layer model parallelism, and it was popularized for LLM training by Megatron-LM [Shoeybi et al. 2019]. The key idea is that the large matrix multiplications inside Transformer layers can be split across multiple GPUs. Rather than having one GPU store and compute an entire weight matrix, each GPU stores a shard and computes part of the result. This makes the model feasible to store and execute even when a single device could not hold the full layer.

Tensor parallelism addresses the model-size problem, but it introduces a new cost: communication moves directly into the forward and backward pass. Instead of performing one major synchronization near the end of a training step, the system may need to exchange partial results repeatedly throughout execution. In other words, model parallelism solves a memory bottleneck by introducing tighter coordination into the critical path. ARK is motivated by exactly this issue, arguing that CPU-driven orchestration can itself become a bottleneck and proposing GPU-driven execution so that scheduling and coordination can occur with less CPU involvement [Hwang et al. 2023].

A second major strategy is pipeline parallelism, represented clearly by PipeDream [Narayanan et al. 2019]. Pipeline parallelism partitions the model by layers into stages, with different groups of GPUs responsible for different segments of the network. If this were done naively, later stages would remain idle while earlier stages finished processing the full batch. What makes pipeline parallelism effective is microbatching. Instead of sending the entire batch through the model at once, the batch is split into smaller microbatches that move through the stages like items on an assembly line. While one stage processes one microbatch, an earlier stage can already begin the next. This allows multiple stages to remain active at the same time and improves hardware utilization.

PipeDream's contribution is not only the use of pipeline parallelism, but also the recognition that pipeline execution is fundamentally a scheduling problem [Narayanan et al. 2019]. The system must decide how many microbatches to keep in flight, how to schedule forward and backward passes, and how to manage weight updates consistently across the pipeline. These are nontrivial systems issues because poor scheduling can either waste hardware or introduce stale updates that complicate training.

Pipeline parallelism also brings its own challenges. First, there are pipeline bubbles: at the beginning and end of execution, some stages remain idle because the pipeline is still filling or draining. Second, the overall speed of the pipeline is limited by its slowest stage. If one stage contains more expensive layers or more communication than the others, it becomes the bottleneck for the entire system. These issues make pipeline parallelism effective, but only when stage partitioning and execution scheduling are handled carefully.

In practice, modern LLM training rarely relies on only one form of parallelism. Instead, most large-scale systems use hybrid parallelism, combining data parallelism, tensor parallelism, and pipeline parallelism because each addresses a different bottleneck. Narayanan et al. show that combining these strategies is essential for scaling large language model training efficiently across GPU clusters [Narayanan et al. 2021]. Data parallelism improves throughput across replicas, tensor parallelism allows large layers to fit across multiple GPUs, and pipeline parallelism distributes deep models across stages. Used together, these methods make it possible to train models that would be impractical under any one strategy alone.

The downside of hybrid parallelism is that it creates a large configuration space. System designers must choose how many data-parallel replicas to use, how large each tensor-parallel group should be, how many pipeline stages to create, and what microbatch size to assign. These choices interact with one another and with cluster properties such as memory capacity, interconnect bandwidth, and topology. As a result, hybrid parallelism is not simply a matter of combining techniques. It becomes a planning and tuning problem. Frameworks such as Alpa are motivated in part by this complexity, aiming to automate the partitioning and scheduling decisions required for efficient distributed training [Zheng et al. 2022].

Recent work has also expanded the design space beyond these three standard strategies. For example, Ring Attention introduces context parallelism, which distributes long input sequences across a ring of GPUs and overlaps communication with computation, enabling context lengths that scale with the number of devices [Liu et al. 2024]. This line of work suggests that parallelism strategies for LLMs are still evolving as model architectures and workload demands continue to change.

Overall, parallelism strategies can be understood as different answers to the same systems question: how should work be divided so that memory limits are respected, communication costs are controlled, and GPUs remain highly utilized? Data parallelism provides a simple and effective baseline, but it breaks down because of memory duplication and communication overhead. Tensor parallelism addresses model size by partitioning computation within layers, but it introduces communication into the critical path. Pipeline parallelism improves utilization for deep models, but it also creates bubble and load-balancing challenges. Hybrid and newer forms of parallelism combine these ideas because no single method is sufficient at LLM scale.

3.2 Memory-Oriented System Optimizations

Memory is often the first reason LLM training fails, and the key point is that the problem is not limited to model weights alone. Training requires several large categories of state to be resident in memory at the same time: weights, which are the parameters being learned; gradients, which are produced during backpropagation and are often roughly the same size as the weights; optimizer state, which stores additional tensors needed by optimizers such as Adam and can require multiple times the weight size; and activations, which are intermediate results from the forward pass that must be retained so the backward pass can compute gradients. A useful intuition is that inference is like reading a book, since only the finished text is needed, whereas training is like writing and editing that book with drafts, annotations, and revisions, which requires much more supporting material.

Memory-oriented system techniques aim to reduce peak memory usage, but they do so in different ways. Some reduce the amount of activation data that must be stored. Others reduce redundancy in optimizer state, gradients, or parameters across GPUs. Still others move parts of the training state to slower but larger memory tiers such as CPU memory or SSDs. Across all of these methods, the basic systems question is the same: how can memory pressure be reduced without introducing so much extra communication or recomputation that training becomes inefficient?

A widely used starting point is activation checkpointing [Chen et al. 2016]. In standard training, many activations from the forward pass are saved so they can be reused during the backward pass. For large models, long sequences, or deep networks, these activations can consume a substantial fraction of GPU memory. Checkpointing reduces this cost by storing only selected activations at designated checkpoints. When the backward pass later needs an activation that was not saved, the system recomputes it by rerunning part of the forward computation. This makes activation checkpointing a direct time-for-memory tradeoff: the system performs extra computation in exchange for lower memory usage. In practice, checkpointing is popular because it is relatively simple to apply and often makes the difference between a model fitting in memory and failing with an out-of-memory error.

Checkpointing helps mainly with activations, but large LLMs often remain bottlenecked by optimizer state and duplication across workers. This is where ZeRO becomes one of the most important ideas in large-scale training [Rajbhandari et al. 2020]. The central observation behind ZeRO is that standard data parallelism wastes memory because every GPU stores a full copy of the optimizer state and gradients, even though those copies are redundant across replicas. ZeRO reduces this redundancy by sharding training state across GPUs, so each worker stores only a fraction rather than the whole. Across the full group, the shards still represent the complete training state, but no single GPU must hold everything at once.

ZeRO is especially useful because it can be applied in stages, which makes the tradeoffs easier to understand:

(1) Stage 1: shard optimizer state
(2) Stage 2: shard optimizer state and gradients
(3) Stage 3: shard optimizer state, gradients, and parameters

As ZeRO moves from Stage 1 to Stage 3, per-GPU memory consumption falls significantly, but the system must perform more coordination to assemble the required state at the right time. In practice, this means more all-gathers and reduce-scatters during training. This is an important systems point: memory optimization is not isolated from communication optimization. Sharding reduces memory pressure, but it also changes the communication pattern of training. If collective operations are slow, aggressively sharding model state can become expensive enough to offset some of the memory gains.

Figure 2 shows this progression visually. Baseline data parallelism replicates the full training state on every GPU, while successive ZeRO stages progressively shard optimizer state, gradients, and parameters. The figure is useful because it highlights that ZeRO is not a single mechanism, but a sequence of increasingly aggressive tradeoffs between memory savings and coordination overhead.

ZeRO-Infinity extends this idea further by introducing memory tiering [Rajbhandari et al. 2021]. GPU memory is fast but limited, CPU memory is much larger but slower, and SSD or NVMe storage offers even greater capacity at still higher latency. ZeRO-Infinity treats these resources as a hierarchy of memory tiers, allowing parts of model state to reside off GPU and be transferred in only when needed. This breaks the assumption that all training state must

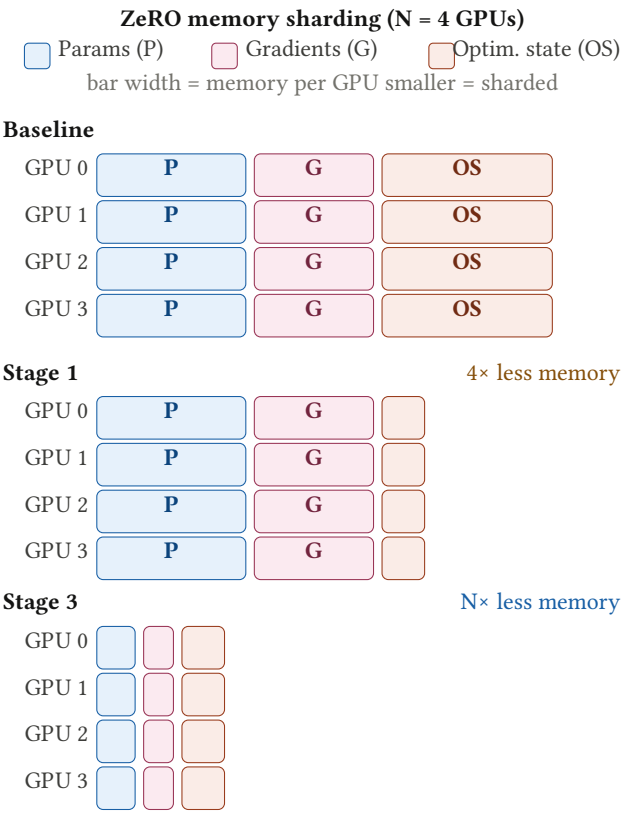

Figure 2. ZeRO sharding. Each stage shards one more component, reducing memory 4× then N× total.

Fig. 2. Memory savings across ZeRO stages compared with baseline data parallelism. In standard data parallelism, each GPU replicates the full training state. ZeRO progressively shards optimizer state, gradients, and parameters across workers, reducing per-GPU memory usage while increasing coordination overhead.

remain in GPU memory at all times, which is crucial for very large models.

The main challenge with offloading is not whether data can be moved, but when it is moved. If the GPU needs a tensor that is still on CPU or SSD, the GPU stalls. If tensors are moved too early or too frequently, the system wastes bandwidth and may interfere with useful computation. Effective offloading is therefore fundamentally a scheduling problem. The runtime must decide what to prefetch, what to evict, and how to overlap transfers with computation so that slower memory tiers expand capacity without becoming the dominant performance bottleneck.

This concern motivates newer work such as Deep Optimizer States, which focuses specifically on optimizer memory and interleaved offloading [Yu et al. 2024]. Optimizer state is often one of the largest components of training memory, especially for adaptive optimizers used in Transformer training. Offloading it can enable larger models or batch sizes, but only if the transfers are carefully interleaved with computation. Naive offloading can severely degrade performance by forcing each training step to wait on CPU-GPU transfers. The broader lesson is that offloading is only effective when it is overlapped, not merely enabled.

A different direction is represented by FlashAttention [Dao et al. 2022]. Rather than redistributing training state across devices or pushing it to slower memory, FlashAttention reduces memory pressure by changing how attention itself is computed. The method tiles attention into blocks that fit within fast on-chip SRAM, reducing the amount of data that must be written to and read from high-bandwidth memory. In this sense, FlashAttention is still a memory-oriented systems optimization, but it operates at a different level: instead of sharding or offloading state, it restructures computation to minimize memory traffic in the first place. This is an important reminder that memory efficiency can come not only from where data is stored, but also from how computation is scheduled relative to the memory hierarchy.

It is also important to recognize that memory methods and communication methods interact strongly. Sharding introduces additional collectives, and offloading introduces additional transfers between memory tiers. As a result, communication improvements can indirectly improve memory efficiency by reducing the overhead of those extra movements. Topology-aware collectives such as TCCL and collective-synthesis systems such as TACCL matter here because faster, better-matched collectives can make ZeRO-style sharding much cheaper in practice. Similarly, methods that overlap or increase concurrency in communication can reduce the effective cost of offloading and sharding by keeping GPUs from sitting idle. In other words, memory optimizations do not just change what fits in memory. They also change the overall execution schedule of training.

These issues become even more important at very large cluster scale. MegaScale shows that once training runs span more than 10,000 GPUs, small inefficiencies can multiply into substantial waste [Jiang et al. 2024]. A minor stall that seems unimportant on eight GPUs can translate into a large number of lost GPU-hours at production scale. This is one reason modern LLM training stacks rarely rely on only one memory technique. Instead, they typically combine activation checkpointing, ZeRO-style sharding, offloading, and careful overlap of communication and computation.

Overall, memory-oriented system optimizations are best understood through a three-way tradeoff: memory capacity, communication cost, and extra computation. Activation checkpointing saves memory by adding recomputation. ZeRO saves memory by reducing redundancy, but adds coordination and collective communication. Offloading expands effective capacity by using slower memory tiers, but only works well when transfers are carefully scheduled and overlapped. FlashAttention reduces memory pressure by restructuring computation around the memory hierarchy. Together, these approaches show that memory is not just a hardware limit. It is a systems design problem shaped by how state is partitioned, moved, recomputed, and scheduled throughout training.

3.3 Communication Optimization

Communication is the glue that holds distributed training together, and at scale it can also become one of its main bottlenecks. In data-parallel training, the most common communication primitive is

AllReduce: after each backward pass, all GPUs must synchronize their gradient tensors. As models grow larger, those gradients grow larger as well. As clusters expand, more nodes must coordinate at every step. The result is that collective communication can end up dominating total training time.

Romero et al. show that this problem is not just about raw network bandwidth. The way the training stack orchestrates, schedules, and buffers communication operations has a major effect on realized throughput across different deep learning frameworks [Romero et al. 2022]. TACCL tackles the problem from the perspective of algorithm synthesis. Rather than relying on a fixed collective algorithm, it generates topology-aware collective schedules by taking the cluster's physical network topology as input and automatically synthesizing an execution plan [Shah et al. 2023]. TCCL pushes this idea further, showing that topology-aware communication can substantially narrow the gap between theoretical and achieved bandwidth in real clusters [Li et al. 2024].

AIACC-Training addresses communication latency by making gradient exchange more concurrent. Instead of waiting for the entire backward pass to finish before beginning synchronization, it overlaps gradient communication with the remaining backward computation using multiple communication streams. This pipelined design can hide a significant fraction of synchronization cost without changing training semantics [Lin et al. 2022].

Gradient compression offers a complementary approach. Instead of communicating the full gradient tensor, workers send a compressed approximation and then correct for the resulting error over later steps. PowerSGD does this through low-rank approximation via power iteration, compressing gradient matrices into two smaller factors [Vogels et al. 2019]. At compression ratios above 128×, PowerSGD reports a 23% reduction in wall-clock time for image classification and a 55% reduction for language modeling, and it has been integrated directly into PyTorch DDP as a standard gradient compression hook [Vogels et al. 2019].

To keep this section clear, it is useful to group these approaches by what they optimize. Topology-aware collectives such as TACCL and TCCL try to make communication itself more efficient by matching algorithms to the cluster. Overlap-oriented approaches such as AIACC-Training try to hide communication behind computation. Compression methods such as PowerSGD try to reduce the amount of data that must be sent in the first place. This makes communication optimization a useful example of a broader systems pattern in LLM training: the same bottleneck can often be attacked by redesigning algorithms, schedules, or data representation.

3.4 Fault Tolerance and Resilience

Fault tolerance becomes essential once training runs stretch across thousands of GPUs and continue for days or weeks. At that scale, failures are not unusual edge cases. They are part of normal operation. Hardware failures, network partitions, and cloud spot-instance preemptions occur regularly in large training environments. MegaScale shows that at more than 10,000 GPUs, even small per-step inefficiencies can accumulate into enormous costs, and hardware failures are a major source of training downtime [Jiang et al. 2024]. A system that cannot recover gracefully from these disruptions must either restart training from scratch or fall back on checkpoint recovery, both of which become increasingly expensive at scale.

The standard recovery mechanism is checkpointing: periodically saving model and optimizer state to persistent storage so that training can resume from a recent point rather than from the beginning. But checkpointing introduces its own tradeoffs. Writing hundreds of gigabytes of state to storage takes time and can interrupt training progress. If checkpoints are too infrequent, more work is lost after a failure. If they are too frequent, the I/O overhead becomes substantial. Asynchronous checkpointing tries to reduce this cost by overlapping state saving with training computation, but it also introduces additional complexity around consistency and recovery correctness.

Bamboo takes a different approach, designed specifically for preemptible cloud instances. Its key insight is that pipeline-parallel training already contains idle time in the form of pipeline bubbles, and those bubbles can be used for redundant computation of neighboring stages. In Bamboo, each node computes not only its own pipeline stage but also a redundant copy of its neighbor's stage. If a node is preempted, its neighbor already holds a valid copy of the lost state. This allows the system to recover without depending entirely on checkpoint reloads from remote storage. Bamboo reports a 2.4 to 3.6 times cost reduction relative to on-demand instances and outperforms checkpoint-based approaches by 3.7 times in throughput [Thorpe et al. 2023].

Oobleck provides a stronger resilience mechanism by introducing pipeline templates, which are precomputed heterogeneous pipeline configurations that can cover the remaining resources after up to f simultaneous failures. The system generates at least $f + 1$ logically equivalent pipeline replicas, so model state is already replicated across those replicas and recovery does not require fetching state from remote storage. A dynamic programming algorithm then finds a valid reconfiguration plan in seconds. In experiments on GPT-2 and GPT-3 variants, Oobleck outperforms Bamboo by up to 13.9 times and Varuna by up to 29.6 times in throughput while still maintaining explicit fault-tolerance guarantees [Jang et al. 2023].

Taken together, these systems highlight two different ways of thinking about resilience. Checkpointing treats failure recovery as something external to the normal execution path: training runs, state is periodically saved, and recovery restores from storage when something goes wrong. Bamboo and Oobleck instead build fault tolerance more directly into execution through redundancy and fast reconfiguration. This makes the broader lesson clear. At LLM scale, resilience cannot be treated as an afterthought. Recovery strategy is part of system design. The question is no longer only how to train efficiently when everything goes right, but how to keep training productive when failures are inevitable.

3.5 Sparse MoE Training

Dense Transformer training improves model capability, but it also increases compute roughly in proportion to model size. Mixture-of-Experts (MoE) architectures change that tradeoff by decoupling parameter count from FLOPs per token. Instead of activating the entire model for every input, MoE models activate only a small subset of expert subnetworks for each token. This makes it possible

to build models with far more parameters without increasing computation proportionally. At the same time, that sparsity introduces a new set of distributed systems challenges.

GShard was one of the earliest systems to address large-scale sharding for MoE Transformers [Lepikhin et al. 2021]. It replaces every other feed-forward layer with a sparsely gated MoE layer and provides lightweight annotation APIs that allow the XLA compiler to partition the model automatically across 2,048 TPU v3 cores. A top-2 gating mechanism, combined with capacity constraints and load-balancing losses, helps manage uneven expert utilization. Using this approach, GShard trained a 600B-parameter model in 4 days, a workload that the authors estimate would have required more than 235 TPU core-years for a comparable dense model [Lepikhin et al. 2021].

The main distributed systems challenge in MoE training is all-to-all communication. In a standard Transformer, communication is usually limited to tensor-parallel AllReduce operations and pipeline-parallel point-to-point transfers. In an MoE model, by contrast, tokens must be routed to their assigned experts, which may reside on different GPUs. That routing pattern naturally maps to all-to-all communication and can dominate total training time, especially at large scale.

Janus addresses this problem through load-aware expert placement and adaptive routing, which reduce all-to-all communication volume [Liu et al. 2023]. ScheMoE takes a complementary approach by introducing pipeline-aware expert scheduling that overlaps expert computation with all-to-all communication in order to hide latency [Shi et al. 2024]. Together, these systems show that MoE training is not just a matter of changing model architecture. It also requires rethinking the surrounding communication and scheduling stack so that sparsity leads to real efficiency gains rather than new bottlenecks.

One reason this section matters is that MoE systems make the connection between architecture and systems especially visible. In dense training, adding parameters usually increases both model size and compute in a fairly direct way. In MoE training, architectural sparsity changes the communication pattern itself. That means systems support is not merely helping a fixed model run faster. It is helping make a different class of models practical at all.

3.6 Inference Serving

Once a model is trained, serving it efficiently introduces a different set of distributed systems problems. Autoregressive generation, where an LLM produces output one token at a time, creates memory, scheduling, and latency challenges that do not appear in the same form during batch training. One of the most important resources during inference is the KV cache: the key and value tensors for all previous tokens in a sequence must remain in GPU memory throughout generation, and their size grows with both sequence length and batch size.

Standard inference systems typically pre-allocate a contiguous block of memory for the maximum possible sequence length of each request. In practice, this wastes 60 to 80% of KV cache memory because of internal and external fragmentation, which in turn limits effective batch size and reduces throughput. vLLM addresses this problem with PagedAttention, an attention mechanism inspired by operating system virtual memory [Kwon et al. 2023]. The KV cache is stored in fixed-size, non-contiguous physical blocks that are addressed through a block table, allowing memory to be allocated dynamically with less than 4% waste. Copy-on-write sharing also allows requests with common prefixes to reuse physical blocks, reducing memory use for beam search by up to 55%. Evaluations on OPT and LLaMA models show that vLLM achieves 2 to 4 times higher throughput than comparable systems at similar latency [Kwon et al. 2023].

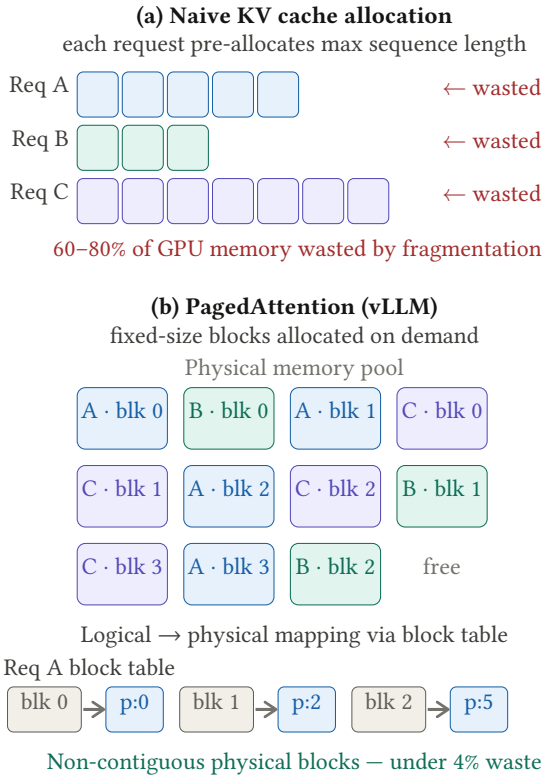

Figure 3. Naive allocation (a) wastes 60–80% of KV cache memory. PagedAttention (b) maps logical blocks to any free physical block, nearly elimin

Fig. 3. PagedAttention in vLLM. Traditional contiguous KV-cache allocation causes substantial internal and external fragmentation, wasting memory and limiting batch size. PagedAttention instead stores KV cache in fixed-size non-contiguous blocks mapped through a block table, greatly reducing memory waste and improving serving throughput.

Figure 3 illustrates this contrast directly. Traditional contiguous allocation wastes memory because each request reserves space for its maximum sequence length, even when much of that space remains unused. PagedAttention avoids this by paging KV cache into fixed-size blocks, allowing memory to be allocated more flexibly and shared more efficiently across requests.

Orca focuses on a different inefficiency: request-level scheduling forces all requests in a batch to finish before new ones can enter,

and it causes short requests to wait behind long ones [Yu et al. 2022]. Orca instead introduces iteration-level scheduling, now more commonly called continuous batching, where the scheduler operates at the granularity of individual token-generation steps and can add or remove requests at every iteration. Selective batching addresses the fact that attention operations often need to process requests individually because KV cache shapes differ across requests, while non-attention operations can still be batched normally. The result is a 36.9 times throughput improvement over NVIDIA FasterTransformer on GPT-3 at comparable latency [Yu et al. 2022]. Continuous batching has since become the standard scheduling approach in LLM serving and is now used by vLLM, TensorRT-LLM, and many production serving systems.

DistServe identifies another important tension in inference serving: the prefill phase, which processes input tokens and is largely compute-bound, and the decoding phase, which generates output tokens and is largely memory-bandwidth-bound, have fundamentally different resource profiles [Zhong et al. 2024]. When these two phases share the same GPUs, they can interfere with one another. In particular, a long prefill can delay all decoding requests in the batch. DistServe addresses this by disaggregating the two phases onto separate GPU pools, each with its own optimized parallelism strategy. KV cache data is transferred from prefill instances to decode instances through NVLink or InfiniBand once prefill is complete. A simulation-based placement algorithm then co-optimizes instance count and parallelism configuration to maximize goodput under SLO constraints. DistServe serves up to 7.4 times more requests or achieves 12.6 times tighter SLO targets than vLLM and DeepSpeed-MII [Zhong et al. 2024].

Taken together, these systems show that inference serving is not simply a smaller or easier version of training. It raises its own systems questions about memory allocation, scheduling granularity, phase interference, and resource disaggregation. As LLM deployment becomes more widespread, these serving-side optimizations increasingly determine whether large models can meet practical latency and throughput requirements in real-world settings.

3.7 Production Infrastructure and Automated Parallelism

The techniques described in the previous sections do not operate in isolation. In real large-scale training runs, systems must combine parallelism strategies, memory optimizations, communication tuning, and fault recovery into a single coordinated infrastructure. MegaScale offers one of the clearest public accounts of what this looks like in practice [Jiang et al. 2024]. In training runs that span more than 10,000 GPUs in a single job, MegaScale identifies the main sources of inefficiency at extreme scale: stragglers caused by hardware variance, redundant collective operations, and the cascading effects of even infrequent hardware failures. The paper describes a set of engineering techniques, including RDMA-optimized communication, operator fusion, and a custom fault detection and recovery pipeline, that together achieve model floating-point utilization (MFU) above 55% at 12,288 GPUs. The central lesson is that no single technique is enough on its own. Production-scale training depends on coordinating improvements across the hardware, communication, runtime, and operations layers at the same time.

Automated parallelism addresses a different but closely related challenge: the configuration space for large-scale training is too large to tune manually. Choosing the right combination of parallelism strategies, pipeline stages, microbatch sizes, and tensor partition degrees quickly becomes impractical as models and clusters grow. Alpa approaches this as a hierarchical two-level search problem, separating inter-operator parallelism such as pipeline stages from intra-operator parallelism such as tensor partitioning [Zheng et al. 2022]. It uses integer linear programming at the inter-operator level and dynamic programming at the intra-operator level. Given a model and a cluster description, Alpa can automatically generate an execution plan without requiring user annotations.

Pathways addresses a deeper structural limitation in existing training frameworks. Many traditional systems assume SPMD execution, where all devices run the same program in lockstep. That model works well for some workloads, but it does not naturally express pipeline parallelism, MoE sparsity, or multi-task computation without substantial additional engineering. Pathways instead introduces an asynchronous distributed dataflow model, built as a sharded graph of asynchronous operators, that separates the control plane from the data plane and enables MPMD execution across multiple accelerator pods [Barham et al. 2022]. On standard SPMD workloads, it achieves utilization comparable to multi-controller systems, while also supporting more flexible execution patterns. It is also the infrastructure used to train PaLM, a 540B-parameter model, on 6,144 TPU v4 chips [Barham et al. 2022].

This final systems theme helps connect the rest of the survey. Parallelism, memory optimization, communication tuning, resilience, and sparse routing may be easier to describe one by one, but production training stacks must combine them all under real cluster constraints. That is why papers such as MegaScale, Alpa, and Pathways are useful capstones. They show not just individual ideas, but how those ideas are coordinated in practice.

4 Synthesis and Cross-Cutting Themes

Across the seven themes surveyed, several broader patterns emerge that help frame how distributed LLM systems should be understood in practice.

The first is that system techniques interact strongly. Sharding optimizer state through ZeRO, for example, reduces memory usage but introduces additional all-gather collectives. If those collectives are slow because the communication algorithm is poorly matched to cluster topology, then memory savings come with a clear communication cost. FlashAttention reduces activation memory, but when it is combined with context parallelism such as Ring Attention, KV blocks still need to move between neighboring devices, which means both memory bandwidth and network bandwidth remain important. Pipeline parallelism introduces pipeline bubbles that can waste computation, but Bamboo shows that those same bubbles can also be repurposed for redundant execution that improves resilience. In other words, no single technique operates in isolation. Each optimization changes the surrounding execution pattern, often in ways that create new tradeoffs elsewhere in the system.

The second observation is that the boundary between training and serving is becoming less sharp. Techniques first developed

for training, such as tensor parallelism and pipeline parallelism, now appear directly in inference systems as well. PagedAttention borrows ideas from operating systems and applies them to inference-time memory management. DistServe's separation of prefill and decoding is, in effect, a new form of pipeline parallelism designed for serving. More broadly, the same core distributed systems ideas, including partitioning, scheduling, resilience, and memory hierarchy management, increasingly apply across both phases. Training and serving differ in workload shape and performance goals, but the systems principles underneath them are becoming more unified.

A third observation is that scale changes which bottlenecks matter. Data parallelism may be sufficient for smaller models, but once a model can no longer fit on a single GPU, tensor parallelism becomes necessary. Once training spans more than a few dozen nodes, collective communication can begin to dominate step time, making topology-aware algorithms and communication overlap much more important. At the scale of thousands of GPUs, hardware failures occur often enough that fault tolerance can no longer be treated as a secondary concern. It becomes a core design requirement, as systems such as Bamboo and Oobleck make clear. MegaScale makes this progression especially concrete: engineering issues that are barely noticeable at 8 GPUs become dominant bottlenecks at 10,000.

The fourth observation is that automation becomes essential as scale increases. The configuration space for hybrid parallelism is large, involving choices about tensor-parallel degree, pipeline stages, data-parallel replicas, and microbatch count. The best setting depends not only on the model, but also on cluster topology, memory capacity, and communication characteristics. These interactions are difficult to reason about manually and even harder to tune by hand. Systems such as Alpa and Pathways represent a broader response to this complexity: instead of treating parallelization strategy as a manual engineering decision, they turn it into a search and optimization problem. As models grow larger and hardware environments become more diverse, this kind of automation will only become more important.

What ties these observations together is that distributed LLM systems are shaped less by any one isolated bottleneck than by interactions across the stack. A change that looks like a local optimization often shifts pressure somewhere else. For that reason, the most useful way to read this literature is not as a list of disconnected tricks, but as a collection of systems responses to tightly coupled constraints in compute, memory, communication, and reliability.

5 Discussion

The papers surveyed here collectively map out the major distributed systems challenges involved in training and serving large language models. Several directions stand out as especially important for future work.

One is heterogeneous hardware. Many current systems assume relatively homogeneous GPU or TPU clusters, where devices have similar compute capability, memory size, and interconnect performance. That assumption becomes harder to maintain as training jobs span multiple cloud regions, combine different GPU generations, or incorporate specialized accelerators. Some existing work points in this direction, including experience with TPU-scale training and multi-pod execution, but heterogeneous GPU clusters remain an open challenge. Future systems will need to reason explicitly about imbalance, placement, and scheduling in environments where not every device behaves the same way.

Another important direction is the co-design of model architecture and distributed systems. Mixture-of-Experts models make this especially clear: architectural decisions about sparse gating immediately create new communication patterns, particularly all-to-all exchanges, which then require new systems support. The same pattern is likely to appear again with future architectural choices involving sparsity, attention design, quantization, or long-context mechanisms. As model design evolves, distributed systems work will increasingly need to anticipate those changes rather than react to them after the fact.

A third area is inference serving at scale, which is still developing rapidly. Important building blocks already exist, including improved scheduling, more efficient KV-cache management, and resource disaggregation. Even so, many questions remain open. Multi-model serving, speculative decoding at scale, and serving across heterogeneous or multi-datacenter environments are still evolving areas. More broadly, inference brings together strict latency targets, unpredictable request patterns, and distributed resource management in ways that make it a particularly rich systems problem.

A final challenge is observability and debugging at scale. Large training and serving systems are difficult not only to optimize, but also to understand when something goes wrong. MegaScale highlights the amount of engineering effort required to identify slow nodes, diagnose communication bottlenecks, and respond quickly to hardware failures. As these systems become more complex, tools for distributed tracing, performance profiling, and anomaly detection will become increasingly important. In practice, a sophisticated training stack is only as usable as the visibility it gives operators into its own behavior.

Across these directions, a common theme is that future progress will likely depend on stronger integration between model design, runtime systems, and operations tooling. The next generation of systems work will not only need to make training and serving faster. It will also need to make them more adaptive, more debuggable, and more robust across increasingly diverse deployment settings.

6 Conclusion

Training and serving large language models is fundamentally a distributed systems problem. The scale of modern LLMs, with billions of parameters, enormous compute demands, and training runs that last for days or weeks, requires solutions that span the full stack. These solutions range from how model state is partitioned across GPU memory hierarchies, to how gradients move across cluster networks, to how inference requests are scheduled and KV cache memory is managed during serving. This survey has traced those solutions through seven major themes: parallelism strategies, memory-oriented optimizations, communication optimization, fault tolerance, sparse MoE systems, inference serving, and production infrastructure and automation.

The clearest lesson across these themes is that distributed systems tradeoffs are rarely isolated. Sharding reduces memory pressure but

increases communication. Compression reduces bandwidth demand but can introduce new concerns around error and convergence. Disaggregated serving can improve latency and throughput, but it also introduces data transfer overhead between phases. Fault tolerance based on redundancy can reduce recovery cost, but it does so by consuming additional compute during normal execution. What matters in practice is not just understanding each of these techniques on its own, but understanding how they interact within a full training or serving stack.

The papers surveyed here represent a strong snapshot of the current state of the art, but the field is moving quickly. New model architectures will continue to create new systems challenges, and the line between training and inference will likely keep blurring as systems reuse ideas across both settings. What will remain constant is the need to reason carefully about the interaction of compute, memory, and communication across many devices. At its core, that is the central question of distributed systems, and it is also what makes large-scale LLM training and serving possible.

References

Paul Barham, Aakanksha Chowdhery, Jeff Dean, Sanjay Ghemawat, et al. 2022. Pathways: Asynchronous Distributed Dataflow for ML. In *Proceedings of Machine Learning and Systems (MLSys)*.

Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. 2016. Training Deep Nets with Sublinear Memory Cost. In *International Conference on Machine Learning (ICML)*.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Changho Hwang, KyoungSoo Park, Ran Shu, Xinyuan Qu, Peng Cheng, and Yongqiang Xiong. 2023. ARK: GPU-driven Code Execution for Distributed Deep Learning. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

Insu Jang, Zhenning Yang, Zhen Zhang, Xin Jin, and Mosharaf Chowdhury. 2023. Oobleck: Resilient Distributed Training of Large Models Using Pipeline Templates. In *ACM Symposium on Operating Systems Principles (SOSP)*.

Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, et al. 2024. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *ACM Symposium on Operating Systems Principles (SOSP)*.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. 2021. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *International Conference on Learning Representations (ICLR)*.

Y. Li et al. 2024. TCCL: Topology-Aware Collective Communication for Deep Learning. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

Lixiang Lin, Shenghao Qiu, Ziqi Yu, Liang You, Long Xin, Xiaoyang Sun, Jie Xu, and Zheng Wang. 2022. AIACC-Training: Optimizing Distributed Deep Learning Training through Multi-streamed and Concurrent Gradient Communications. In *International Conference on Distributed Computing Systems (ICDCS)*.

Hao Liu, Matei Zaharia, and Pieter Abbeel. 2024. Ring Attention with Blockwise Transformers for Near-Infinite Context. In *International Conference on Learning Representations (ICLR)*.

Y. Liu et al. 2023. Janus: Efficient and Accurate Distributed Training of Mixture-of-Experts Models. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. PipeDream: Generalized Pipeline Parallelism for DNN Training. In *ACM Symposium on Operating Systems Principles (SOSP)*.

Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Anand Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, Amar Phanishayee, and Matei Zaharia. 2021. Efficient Large-Scale Language Model Training on GPU Clusters Using Hybrid Parallelism. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme-Scale Deep Learning. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

Joshua Romero, Junqi Yin, Nouamane Laanait, Bing Xie, M. Todd Young, et al. 2022. Accelerating Collective Communication in Data Parallel Training across Deep Learning Frameworks. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

Alexander Sergeev and Mike Del Balso. 2018. Horovod: Fast and Easy Distributed Deep Learning in TensorFlow. *arXiv preprint arXiv:1802.05799* (2018).

Aashaka Shah et al. 2023. TACCL: Guiding Collective Algorithm Synthesis using Communication Sketches. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

Y. Shi et al. 2024. ScheMoE: Efficient Mixture-of-Experts Training with Scheduling and Communication Optimizations. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. In *International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*.

John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. 2023. Bamboo: Making Preemptible Instances Resilient for Affordable Training of Large DNNs. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

Thijs Vogels, Sai Praneeth Karimireddy, and Martin Jaggi. 2019. PowerSGD: Practical Low-Rank Gradient Compression for Distributed Optimization. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

X. Yu et al. 2024. Deep Optimizer States: Scalable Training of Transformer Models Using Interleaved Offloading. In *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*.

Lianmin Zheng, Zhiqiang Li, Hao Zhang, Yonghao Zhuang, Zhen Chen, Yujia Huang, Yao Wang, Yida Xu, Donglin Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.

Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.